UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN



MODELO FÍSICO DE LA BASE DE DATOS COMPUTECH

ESTRUCTURA, VISTAS, PROCEDIMIENTOS, CONSULTAS Y EXPORTACIÓN INTEGRAL

ACTIVIDAD 01 | SEMANA 06

ALUMNO:

➢ **QUISPE TICA RUTH NAYELI**

CURSO:

➢ **DESARROLLO DE APPS**

CICLO:

➢ **IV**

SECCIÓN:

➢ **A1**

HUANCAYO - PERÚ
2026

Contenido

Presentación

1. Objetivos

2. Ficha técnica del entorno

3. Estructura general de CompuTechDB

4. Diccionario de datos y evidencias

5. Relaciones, restricciones y consistencia

6. Vistas de la base de datos

7. Procedimientos almacenados (7.1 Operaciones CRUD)

8. Consultas y resultados de comprobación

9. Procedimiento para generar el archivo SQL completo

10. Validación del SQL obtenido

11. Resultados y conclusiones

12. Recomendaciones para uso académico

Presentación

CompuTech es una tienda web de computadoras y accesorios desarrollada con Java, JSP, Servlets y Maven, bajo arquitectura MVC. En la etapa inicial los productos vivían en memoria (ArrayList); en esta etapa se incorpora una base de datos relacional que da persistencia a los productos, usuarios, carritos y pedidos de la aplicación.

Este informe explica la base de datos CompuTechDB, creada con el script CompuTechDB.sql: qué tablas la componen, qué guarda cada columna, cómo se relacionan las tablas y qué reglas garantizan que los datos sean coherentes.

Criterio de presentación: se utiliza T-SQL sencillo y verificable (USE, GO y SELECT) como procedimiento didáctico. En este trabajo se consultan únicamente las tablas reales de CompuTechDB, tal como las crea el script. Las operaciones de modificación (CREATE, INSERT, UPDATE, DELETE – CRUD) se describen para explicar el funcionamiento de la aplicación y se distinguen siempre de lo que el script ya contiene.

1. Objetivos

Objetivo general. Documentar y sustentar el diseño físico de la base de datos CompuTechDB en SQL Server, demostrando que respalda las funciones de la aplicación web CompuTech (catálogo, cuentas de usuario, carrito y pedidos).

Objetivos específicos

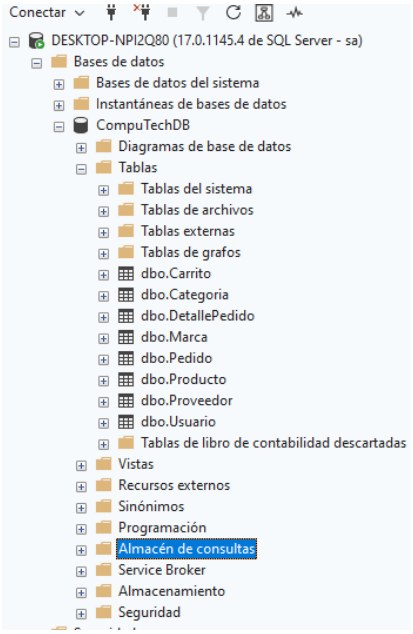
- Describir las 8 tablas del modelo físico: Categoria, Marca, Proveedor, Producto, Usuario, Carrito, Pedido y DetallePedido.
- Elaborar el diccionario de datos con tipo, nulabilidad y restricciones de cada columna.
- Explicar las claves primarias, claves foráneas y reglas de integridad (UNIQUE, CHECK, DEFAULT, ON DELETE CASCADE).
- Presentar consultas de comprobación que permitan verificar que la base se creó correctamente.
- Proponer vistas y procedimientos almacenados que simplifiquen el acceso desde la aplicación.

2. Ficha técnica del entorno

Elemento	Detalle
Nombre de la base de datos	CompuTechDB
Gestor	Microsoft SQL Server
Esquema	dbo
Archivo de origen	CompuTechDB.sql
Número de tablas	8
Re-ejecutable	Sí: si la base existe, la pone en modo SINGLE_USER, la elimina y la vuelve a crear
Aplicación cliente	CompuTech (Java 17+, JSP, Servlets, Maven, Tomcat 11)
Acceso desde la aplicación	Ventana de ingreso (login) conectada a la tabla Usuario
Seguridad de contraseñas	Se guarda un hash PBKDF2 (sal:hash en Base64), nunca la clave en texto plano

El script inicia con USE master, elimina la base anterior con ALTER DATABASE ... SET SINGLE_USER WITH ROLLBACK IMMEDIATE seguido de DROP DATABASE, y crea CompuTechDB desde cero. Esto permite repetir la instalación sin errores durante las pruebas académicas.

3. Estructura general de CompuTechDB

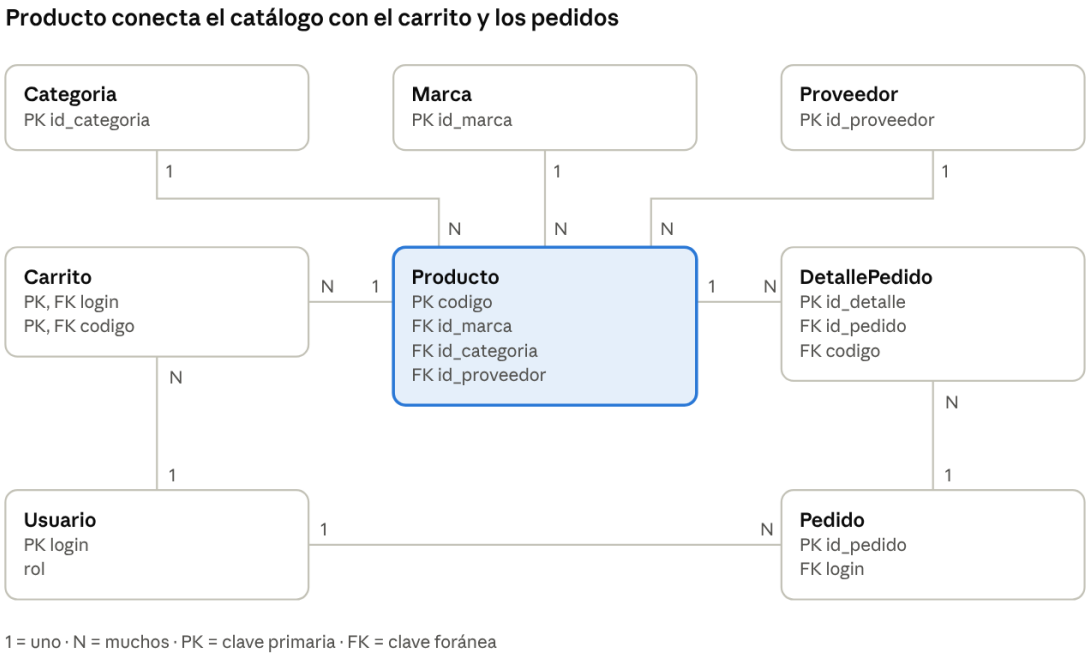


Las 8 tablas se agrupan en tres bloques funcionales:

- **Catálogo (tablas de apoyo y producto):** Categoria, Marca, Proveedor y Producto. Las tres primeras son tablas maestras; Producto las referencia.
- **Seguridad:** Usuario, con tres roles: ADMIN (control total), ALMACENERO (solo stock y estado) y USUARIO (cliente).
- **Ventas:** Carrito (compra en curso), Pedido (cabecera de la compra) y DetallePedido (productos de cada pedido).

Mapa de relaciones (1 a muchos):

- Categoria 1 → N Producto
- Marca 1 → N Producto (opcional)
- Proveedor 1 → N Producto (opcional)
- Usuario 1 → N Carrito, y Usuario 1 → N Pedido
- Producto 1 → N Carrito, y Producto 1 → N DetallePedido
- Pedido 1 → N DetallePedido



Carrito es además la tabla intermedia de la relación muchos a muchos entre Usuario y Producto (un usuario puede tener varios productos en su carrito y un producto puede estar en el carrito de varios usuarios). DetallePedido cumple el mismo papel entre Pedido y Producto.

Datos iniciales que carga el script: 8 categorías, 9 marcas, 4 proveedores, 12 productos y 2 usuarios (un administrador y un almacenero). Carrito, Pedido y DetallePedido quedan vacías: se llenan cuando los clientes usan la aplicación.

4. Diccionario de datos y evidencias

4.1 Categoria

Clasifica los productos (Laptops, Computadoras, Monitores, Periféricos, Componentes, Almacenamiento, Redes, Audio).

Columna	Tipo	Nulo	Restricción / descripción
id_categoria	INT IDENTITY(1,1)	No	PK_Categoria. Código autonumérico
nombre	NVARCHAR(50)	No	UQ_Categoria_nombre: no se repite

4.2 Marca

Fabricante del producto (ASUS, Corsair, Gamer Machine, Kingston, Lenovo, Logitech, MSI, Redragon, Teros).

Columna	Tipo	Nulo	Restricción / descripción
id_marca	INT IDENTITY(1,1)	No	PK_Marca

Columna	Tipo	Nulo	Restricción / descripción
nombre	NVARCHAR(60)	No	UQ_Marca_nombre: no se repite

4.3 Proveedor

Empresa que abastece los productos.

Columna	Tipo	Nulo	Restricción / descripción
id_proveedor	INT IDENTITY(1,1)	No	PK_Proveedor
nombre	NVARCHAR(100)	No	Razón social
ruc	VARCHAR(11)	Sí	UQ_Proveedor_ruc: RUC único (11 dígitos)
telefono	VARCHAR(20)	Sí	Teléfono de contacto
correo	NVARCHAR(120)	Sí	Correo de contacto

Proveedores cargados: Tecno Import S.A.C., Distribuidora Periféricos Perú, Gamer Mayorista E.I.R.L. y Componentes del Centro S.R.L.

4.4 Producto

Tabla central del catálogo. Cada fila es un artículo a la venta.

Columna	Tipo	Nulo	Restricción / descripción
codigo	VARCHAR(10)	No	PK_Producto (P001, P002, …)
nombre	NVARCHAR(120)	No	Nombre comercial
id_marca	INT	Sí	FK_Producto_Marca → Marca
id_categoria	INT	No	FK_Producto_Categoria → Categoria
id_proveedor	INT	Sí	FK_Producto_Proveedor → Proveedor
precio	DECIMAL(10,2)	No	CK_Producto_precio: debe ser mayor que 0
stock	INT	No	CK_Producto_stock: no puede ser negativo
estado	VARCHAR(20)	No	DEFAULT 'Disponible'; CK_Producto_estado: solo 'Disponible' o 'Agotado'
descripcion	NVARCHAR(500)	No	DEFAULT ''
imagen	VARCHAR(100)	No	DEFAULT 'mouse.svg'; nombre del archivo de imagen

Nota de diseño: el estado se guarda tal como lo elige el administrador; cuando el stock es 0 la aplicación lo fuerza a 'Agotado'. En los datos iniciales, P012 (Laptop Lenovo IdeaPad 3 15" Ryzen 5) tiene stock 0 y estado Agotado. Se crean además los índices IX_Producto_categoria e IX_Producto_marca para acelerar los filtros del catálogo por categoría y marca.

4.5 Usuario

Columna	Tipo	Nulo	Restricción / descripción
login	VARCHAR(20)	No	PK_Usuario: nombre de acceso único
clave_hash	VARCHAR(200)	No	Hash PBKDF2 en formato sal(Base64):hash(Base64)
nombre	NVARCHAR(100)	No	Nombre completo
correo	NVARCHAR(120)	No	Correo electrónico
rol	VARCHAR(10)	No	CK_Usuario_rol: ADMIN, ALMACENERO o USUARIO

El script crea dos usuarios iniciales: admin (ADMIN) y almacen (ALMACENERO). Los clientes se registran desde la propia página con el rol USUARIO. Las contraseñas iniciales deben cambiarse tras la instalación.

4.6 Carrito

Guarda un renglón por usuario y producto.

Columna	Tipo	Nulo	Restricción / descripción
login	VARCHAR(20)	No	FK → Usuario, ON DELETE CASCADE
codigo	VARCHAR(10)	No	FK → Producto, ON DELETE CASCADE
cantidad	INT	No	CK_Carrito_cantidad: mayor que 0
fecha	DATETIME2	No	DEFAULT SYSDATETIME()

La clave primaria es compuesta (login, codigo): un producto no puede aparecer dos veces en el carrito de la misma persona; si lo vuelve a agregar, se aumenta la cantidad.

4.7 Pedido

Cabecera de cada compra confirmada.

Columna	Tipo	Nulo	Restricción / descripción
id_pedido	INT IDENTITY(1,1)	No	PK_Pedido
login	VARCHAR(20)	No	FK_Pedido_Usuario → Usuario
fecha	DATETIME2	No	DEFAULT SYSDATETIME()
total	DECIMAL(12,2)	No	CK_Pedido_total: mayor o igual a 0
estado	VARCHAR(20)	No	DEFAULT 'Pendiente'; solo Pendiente, Pagado, Enviado o Cancelado

4.8 DetallePedido

Productos de cada pedido. Guarda el precio del momento de la compra.

Columna	Tipo	Nulo	Restricción / descripción
id_detalle	INT IDENTITY(1,1)	No	PK_DetallePedido
id_pedido	INT	No	FK_Detalle_Pedido → Pedido, ON DELETE CASCADE
codigo	VARCHAR(10)	No	FK_Detalle_Producto → Producto
cantidad	INT	No	CK_Detalle_cantidad: mayor que 0
precio_unitario	DECIMAL(10,2)	No	CK_Detalle_precio: mayor que 0

La restricción UQ_Detalle (id_pedido, codigo) impide repetir el mismo producto dentro de un pedido.

Evidencia. Al ejecutar el script, la primera consulta de verificación lista las tablas base de INFORMATION_SCHEMA.TABLES y debe mostrar exactamente 8 filas: Carrito, Categoria, DetallePedido, Marca, Pedido, Producto, Proveedor y Usuario. La segunda consulta lista los 12 productos con su marca, categoría, precio, stock y estado.

5. Relaciones, restricciones y consistencia

5.1 Claves foráneas

Tabla hija	Columna	Tabla padre	Comportamiento al borrar el padre
Producto	id_marca	Marca	Se impide borrar la marca si tiene productos
Producto	id_categoria	Categoria	Se impide borrar la categoría si tiene productos
Producto	id_proveedor	Proveedor	Se impide borrar el proveedor si tiene productos
Carrito	login	Usuario	CASCADE: se borra su carrito
Carrito	codigo	Producto	CASCADE: el producto sale de los carritos
Pedido	login	Usuario	Se impide borrar un usuario con pedidos (conserva el historial)
DetallePedido	id_pedido	Pedido	CASCADE: al borrar el pedido se borran sus detalles
DetallePedido	codigo	Producto	Se impide borrar un producto vendido (conserva el historial)

5.2 Reglas de integridad

- **Integridad de entidad:** toda tabla tiene clave primaria; Carrito usa clave compuesta.
- **Integridad referencial:** las 8 claves foráneas evitan productos sin categoría, pedidos sin usuario o detalles sin producto.
- **Integridad de dominio (CHECK):** precio mayor que 0, stock mayor o igual a 0, estado y rol dentro de valores permitidos, cantidades positivas y totales no negativos.

- **Unicidad (UNIQUE):** nombres de categoría y marca, RUC del proveedor y producto único por pedido.
- **Valores por defecto (DEFAULT):** estado inicial 'Disponible' en productos y 'Pendiente' en pedidos, fechas con SYSDATETIME(), imagen y descripción predeterminadas.

5.3 Consistencia del negocio

El precio unitario se copia en DetallePedido para que un cambio posterior del precio del producto no altere pedidos antiguos. Los borrados restringidos protegen el historial de ventas, y los borrados en cascada limpian datos temporales (carritos) y dependientes (detalles).

6. Vistas de la base de datos

El script CompuTechDB.sql no define vistas: todas las consultas se hacen directamente sobre las tablas. Para simplificar el acceso se proponen dos vistas, cuyo código está en el Anexo B:

- **vw_CatalogoProductos:** une Producto, Categoria y Marca para mostrar el catálogo con nombres en lugar de códigos.
- **vw_DetallePedidoCompleto:** une Pedido, DetallePedido y Producto para mostrar el comprobante con subtotal por línea.

7. Procedimientos almacenados

El script original tampoco contiene procedimientos almacenados; la aplicación Java ejecuta las sentencias SQL desde su capa de acceso a datos. Como mejora académica se propone el procedimiento usp_ProductosPorCategoria (Anexo B).

7.1 Operaciones CRUD implementadas en CompuTech

Operación	Sentencia	Uso en la aplicación
Crear (Create)	INSERT	Registrar usuarios, agregar productos (ADMIN), agregar al carrito, generar pedidos
Leer (Read)	SELECT	Catálogo, detalle de producto, login, historial de pedidos
Actualizar (Update)	UPDATE	Editar productos (ADMIN); ajustar stock y estado (ADMIN y ALMACENERO); cambiar cantidad del carrito y estado del pedido
Eliminar (Delete)	DELETE	Quitar productos del carrito y eliminar productos del catálogo (ADMIN)

Los permisos por rol son: ADMIN realiza todo el CRUD; ALMACENERO solo actualiza stock y estado; USUARIO solo consulta el catálogo y gestiona su carrito y sus pedidos.

8. Consultas y resultados de comprobación

Todas las consultas usan únicamente tablas reales de CompuTechDB.

Consulta 1 – Tablas creadas (resultado esperado: 8 filas)

```
USE CompuTechDB;
GO
SELECT TABLE_NAME AS tabla
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE'
ORDER BY TABLE_NAME;
```

Consulta 2 – Catálogo completo (resultado esperado: 12 productos, de P001 a P012)

```
SELECT p.codigo, p.nombre, m.nombre AS marca, c.nombre AS categoria,
      p.precio, p.stock, p.estado
FROM dbo.Producto p
INNER JOIN dbo.Categoria c ON c.id_categoria = p.id_categoria
LEFT  JOIN dbo.Marca m     ON m.id_marca = p.id_marca
ORDER BY p.codigo;
```

Se usa INNER JOIN con Categoria porque id_categoria es obligatorio, y LEFT JOIN con Marca porque id_marca admite nulos.

Consulta 3 – Usuarios y roles (resultado esperado: admin y almacen)

```
SELECT login, nombre, correo, rol FROM dbo.Usuario;
```

Consulta 4 – Productos por categoría (resultado esperado: Laptops 2, Periféricos 4 entre otras)

```
SELECT c.nombre AS categoria, COUNT(*) AS cantidad_productos
FROM dbo.Producto p
INNER JOIN dbo.Categoria c ON c.id_categoria = p.id_categoria
GROUP BY c.nombre
ORDER BY cantidad_productos DESC;
```

Consulta 5 – Productos agotados (resultado esperado: P012)

```
SELECT codigo, nombre, stock, estado
FROM dbo.Producto
WHERE estado = 'Agotado' OR stock = 0;
```

Consulta 6 – Valor del inventario por categoría

```
SELECT c.nombre AS categoria, SUM(p.precio * p.stock) AS valor_inventario
FROM dbo.Producto p
INNER JOIN dbo.Categoria c ON c.id_categoria = p.id_categoria
GROUP BY c.nombre
ORDER BY valor_inventario DESC;
```

Consulta 7 – Productos con su proveedor

```
SELECT p.codigo, p.nombre, pr.nombre AS proveedor
FROM dbo.Producto p
LEFT JOIN dbo.Proveedor pr ON pr.id_proveedor = p.id_proveedor
ORDER BY pr.nombre, p.codigo;
```

9. Procedimiento para generar el archivo SQL completo

1. Abrir SQL Server Management Studio (SSMS) y conectarse a la instancia del equipo.
2. Abrir el archivo CompuTechDB.sql (Archivo → Abrir → Archivo).
3. Ejecutar todo el script con F5. Elimina la base anterior si existe, crea CompuTechDB, las 8 tablas, los índices y carga los datos iniciales.
4. Revisar el panel de resultados: aparecen la lista de tablas y el listado de productos.
5. Al final del script hay SELECT sobre Usuario, Categoria, Marca y Producto que muestran el contenido de cada tabla.

6. Para obtener el script desde una base ya existente: clic derecho en CompuTechDB → Tareas → Generar scripts, y elegir esquema y datos.

El uso de GO separa lotes, de modo que cada tabla se crea en su propio lote y un error se identifica con facilidad.

10. Validación del SQL obtenido

Aspecto validado	Cómo se comprueba	Resultado esperado
Creación de la base	SELECT DB_ID('CompuTechDB')	Devuelve un número, no NULL
Número de tablas	Consulta 1	8 tablas
Datos de catálogo	COUNT(*) en Categoria, Marca, Proveedor, Producto	8, 9, 4 y 12 filas
Usuarios iniciales	Consulta 3	2 usuarios (ADMIN y ALMACENERO)
Regla de precio	INSERT con precio 0	Error por CK_Producto_precio
Regla de stock	UPDATE con stock negativo	Error por CK_Producto_stock
Regla de estado	INSERT con estado 'Otro'	Error por CK_Producto_estado
Clave foránea	INSERT en Producto con id_categoria inexistente	Error por FK_Producto_Categoria
Unicidad	INSERT de una categoría repetida	Error por UQ_Categoria_nombre
Re-ejecución	Ejecutar el script dos veces	Termina sin errores

Las pruebas negativas (las que deben fallar) se hacen dentro de una transacción con ROLLBACK para no alterar los datos.

11. Resultados y conclusiones

- CompuTechDB cumple el modelo físico de 8 tablas y respalda las tres áreas de la aplicación: catálogo, seguridad y ventas.
- El diseño está normalizado: marca, categoría y proveedor se guardan una sola vez en tablas maestras y se referencian desde Producto, evitando repetir información.
- Las restricciones PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK y DEFAULT mantienen la consistencia aunque la aplicación falle, porque las reglas viven en la base de datos.
- La separación Pedido / DetallePedido, con precio_unitario guardado, conserva el historial de ventas correcto aunque cambien los precios.
- La tabla Usuario con rol y contraseña cifrada con PBKDF2 permite el login con permisos diferenciados (ADMIN, ALMACENERO, USUARIO) sin exponer claves.
- El script es re-ejecutable y verificable, lo cual facilita la instalación y la revisión académica.

12. Recomendaciones para uso académico

- Ejecutar el script en una instancia de pruebas, porque elimina la base CompuTechDB si ya existe.

- Cambiar las contraseñas de los usuarios iniciales y no publicarlas en el informe.
- Presentar capturas de la ejecución (lista de 8 tablas y catálogo de 12 productos) como evidencia.
- Acompañar el informe con el modelo conceptual E-R y el diagrama de la base de datos generado en SSMS.
- Usar un usuario de SQL Server con permisos limitados para la conexión de la aplicación, no la cuenta sa.
- En la aplicación Java, usar PreparedStatement para evitar inyección SQL.
- Como mejora futura, implementar las vistas y el procedimiento del Anexo B.